

Atividade — Modelagem MongoDB do SAA

Projeto: SAA — Sistema de Ausência de Alunos

Banco: MongoDB

Integrantes/entrega: preencher conforme a equipe

1. Descrição do negócio

1. O SAA é um sistema escolar para registrar e acompanhar ausências de alunos.
2. A professora identifica quem não entrou em sala e informa os nomes à AOE pelo canal interno da escola.
3. A AOE acessa o sistema e localiza a turma e o aluno.
4. Em seguida, registra a ausência com a data e informações complementares.
5. O sistema localiza os responsáveis vinculados ao aluno.
6. Para cada responsável com e-mail válido, prepara uma comunicação de ausência.
7. A mensagem contém o nome do aluno, a data da ausência e um link seguro.
8. O link leva ao formulário público de justificativa, sem exigir login dos pais.
9. O responsável seleciona um motivo e pode explicar a situação quando escolhe “Outro” .
10. A justificativa fica vinculada à ocorrência correta de ausência.
11. A coordenação consulta pendências, respostas, histórico e indicadores de frequência.
12. O sistema também registra o andamento do e-mail, como pendente, enviado, entregue, aberto ou falho.
13. Os usuários internos são AOE, coordenação e administrador, cada um com acesso controlado.
14. O banco precisa preservar o histórico mesmo quando o aluno muda de turma.
15. As informações principais são alunos, matrículas, turmas, responsáveis, frequências, justificativas e notificações.

2. Principais consultas e operações

As três operações marcadas com **[FREQUENTE]** são as mais usadas no fluxo principal da aplicação.

1. **[FREQUENTE] Localizar um aluno e carregar suas faltas e responsáveis com e-mail válido.**

2. **[FREQUENTE]** Registrar uma nova ausência para uma matrícula em determinada data.
3. **[FREQUENTE]** Gerar o dashboard com faltantes por turma e médias de ausência por mês/ano.
4. Enviar ou colocar na fila uma notificação para o responsável do aluno.
5. Consultar o histórico de notificações, incluindo entrega, abertura e falha.
6. Registrar uma justificativa usando o link seguro recebido por e-mail.
7. Consultar justificativas pendentes e respondidas pela coordenação.
8. Cadastrar ou atualizar aluno, turma, responsável e matrícula.
9. Consultar o histórico de frequência de um aluno sem perder dados de mudanças de turma.
10. Comparar a frequência atual com o percentual mínimo configurado para o período.

3. Coleções e estrutura dos documentos

3.1 Coleção `students`

O que guarda: cadastro básico e estável do aluno.

Campo	Tipo	Descrição
<code>_id</code>	ObjectId	Identificador do aluno.
<code>name</code>	string	Nome completo.
<code>school_id</code>	string	Matrícula ou identificador escolar.
<code>active</code>	boolean	Indica se o cadastro está ativo.
<code>created_at</code>	date	Data de criação.
<code>updated_at</code>	date	Data da última atualização.

Exemplo válido em Extended JSON:

JSON

```
{
  "_id": { "$oid": "507f1f77bcf86cd799439011" },
  "name": "Ana Beatriz Souza",
```

```

"school_id": "MAT-001",
"active": true,
"created_at": { "$date": "2026-02-01T12:00:00Z" },
"updated_at": { "$date": "2026-10-09T12:00:00Z" }
}

```

3.2 Coleção `enrollments`

O que guarda: o vínculo histórico do aluno com uma turma e um período letivo.

Campo	Tipo	Descrição
<code>_id</code>	ObjectId	Identificador da matrícula.
<code>student_id</code>	ObjectId	Referência a <code>students._id</code> .
<code>class_id</code>	ObjectId	Referência a <code>classes._id</code> .
<code>academic_period_id</code>	ObjectId	Referência ao período letivo.
<code>start_date</code>	date	Início do vínculo.
<code>end_date</code>	date/null	Fim do vínculo, quando existir.
<code>active</code>	boolean	Indica matrícula vigente.

Exemplo válido em Extended JSON:

JSON

```

{
  "_id": { "$oid": "507f1f77bcf86cd799439021" },
  "student_id": { "$oid": "507f1f77bcf86cd799439011" },
  "class_id": { "$oid": "507f1f77bcf86cd799439031" },
  "academic_period_id": { "$oid": "507f1f77bcf86cd799439041" },
  "start_date": { "$date": "2026-02-01T00:00:00Z" },
  "end_date": null,
  "active": true
}

```

3.3 Coleção `attendance_records`

O que guarda: a frequência diária oficial por matrícula.

Campo	Tipo	Descrição
-------	------	-----------

<code>_id</code>	ObjectId	Identificador da frequência.
<code>enrollment_id</code>	ObjectId	Referência a <code>enrollments._id</code> .
<code>date</code>	date	Data da frequência.
<code>status</code>	string	<code>PRESENT</code> , <code>ABSENT</code> , <code>EXCUSED</code> OU <code>NOT_REGISTERED</code> .
<code>registered_by</code>	ObjectId	Referência a <code>users._id</code> .
<code>details</code>	objeto/subdocumento	Informações complementares da ocorrência.
<code>details.source</code>	string	Origem da informação, por exemplo <code>INTERNAL_MESSAGE</code> .
<code>details.note</code>	string	Observação da AOE.
<code>created_at</code>	date	Data do lançamento.

Neste documento há um **subdocumento embutido**: `details`.

Exemplo válido em Extended JSON:

JSON

```
{
  "_id": { "$oid": "507f1f77bcf86cd799439051" },
  "enrollment_id": { "$oid": "507f1f77bcf86cd799439021" },
  "date": { "$date": "2026-10-09T00:00:00Z" },
  "status": "ABSENT",
  "registered_by": { "$oid": "507f1f77bcf86cd799439091" },
  "details": {
    "source": "INTERNAL_MESSAGE",
    "note": "Ausência informada pela professora no grupo interno."
  },
  "created_at": { "$date": "2026-10-09T12:00:00Z" }
}
```

3.4 Coleção `guardians`

O que guarda: cadastro dos pais ou responsáveis e seus e-mails.

Campo	Tipo	Descrição
<code>_id</code>	ObjectId	Identificador do responsável.

name	string	Nome completo.
email	string	E-mail informado.
email_normalized	string	E-mail em minúsculas para busca e unicidade.
active	boolean	Indica se pode receber comunicação.

Exemplo válido em Extended JSON:

JSON
<pre>{ "_id": { "\$oid": "507f1f77bcf86cd799439061" }, "name": "Carlos Souza", "email": "Carlos.Souza@exemplo.com", "email_normalized": "carlos.souza@exemplo.com", "active": true }</pre>

3.5 Coleção student_guardians

O que guarda: a relação entre alunos e responsáveis. Ela resolve o relacionamento muitos-para-muitos.

Campo	Tipo	Descrição
_id	ObjectId	Identificador do vínculo.
student_id	ObjectId	Referência a students._id .
guardian_id	ObjectId	Referência a guardians._id .
relationship	string	Exemplo: PAI , MAE ou RESPONSÁVEL .
is_primary	boolean	Indica responsável principal.
receives_absence_notifications	boolean	Permite controlar o recebimento.
active	boolean	Indica vínculo ativo.

Exemplo válido em Extended JSON:

JSON

```
{
  "_id": { "$oid": "507f1f77bcf86cd799439071" },
  "student_id": { "$oid": "507f1f77bcf86cd799439011" },
  "guardian_id": { "$oid": "507f1f77bcf86cd799439061" },
  "relationship": "PAI",
  "is_primary": true,
  "receives_absence_notifications": true,
  "active": true
}
```

3.6 Coleção `email_notifications`

O que guarda: o histórico de cada comunicação e os eventos retornados pelo provedor.

Este documento possui referências entre collections e também um **array de subdocumentos** em `events`.

Campo	Tipo	Descrição
<code>_id</code>	ObjectId	Identificador da notificação.
<code>student_id</code>	ObjectId	Referência a <code>students._id</code> .
<code>attendance_id</code>	ObjectId	Referência a <code>attendance_records._id</code> .
<code>guardian_id</code>	ObjectId	Referência a <code>guardians._id</code> .
<code>recipient_email</code>	string	Snapshot do e-mail usado no envio.
<code>subject</code>	string	Assunto enviado.
<code>body</code>	string	Conteúdo final enviado.
<code>justification_url</code>	string	Link individual da justificativa.
<code>status</code>	string	<code>PENDING</code> , <code>SENT</code> , <code>DELIVERED</code> , <code>OPENED</code> ou <code>FAILED</code> .
<code>provider_message_id</code>	string/null	Identificador do provedor.
<code>events</code>	array de objetos	Eventos de envio, entrega, abertura e falha.

created_by	ObjectId	Referência a <code>users._id</code> .
created_at	date	Criação da notificação.

Exemplo válido em Extended JSON:

```

JSON

{
  "_id": { "$oid": "507f1f77bcf86cd799439081" },
  "student_id": { "$oid": "507f1f77bcf86cd799439011" },
  "attendance_id": { "$oid": "507f1f77bcf86cd799439051" },
  "guardian_id": { "$oid": "507f1f77bcf86cd799439061" },
  "recipient_email": "carlos.souza@exemplo.com",
  "subject": "Comunicado de ausência escolar",
  "body": "Olá, Carlos. Registramos uma ausência de Ana Beatriz Souza em 09/10/2026.",
  "justification_url": "https://saa.exemplo.com/justify/token-temporario",
  "status": "OPENED",
  "provider_message_id": "provider-123456",
  "events": [
    {
      "type": "SENT",
      "occurred_at": { "$date": "2026-10-09T12:01:00Z" },
      "provider_message_id": "provider-123456"
    },
    {
      "type": "DELIVERED",
      "occurred_at": { "$date": "2026-10-09T12:01:04Z" },
      "provider_message_id": "provider-123456"
    },
    {
      "type": "OPENED",
      "occurred_at": { "$date": "2026-10-09T13:20:00Z" },
      "provider_message_id": "provider-123456"
    }
  ],
  "created_by": { "$oid": "507f1f77bcf86cd799439091" },
  "created_at": { "$date": "2026-10-09T12:00:00Z" }
}

```

3.7 Coleção `justifications`

O que guarda: a resposta enviada pelo responsável para uma ocorrência.

Campo	Tipo	Descrição
-------	------	-----------

<code>_id</code>	ObjectId	Identificador da justificativa.
<code>attendance_id</code>	ObjectId	Referência a <code>attendance_records._id</code> .
<code>guardian_id</code>	ObjectId	Referência a <code>guardians._id</code> .
<code>reason_code</code>	string	Código do motivo escolhido.
<code>explanation</code>	string/null	Obrigatória quando o motivo for <code>OTHER</code> .
<code>status</code>	string	<code>PENDING</code> ou <code>RESPONDED</code> .
<code>submitted_at</code>	date/null	Data e hora do envio.
<code>created_at</code>	date	Criação do registro.

Exemplo válido em Extended JSON:

JSON

```
{
  "_id": { "$oid": "507f1f77bcf86cd7994390a1" },
  "attendance_id": { "$oid": "507f1f77bcf86cd799439051" },
  "guardian_id": { "$oid": "507f1f77bcf86cd799439061" },
  "reason_code": "MEDICAL",
  "explanation": "Consulta médica no período da manhã.",
  "status": "RESPONDED",
  "submitted_at": { "$date": "2026-10-09T18:30:00Z" },
  "created_at": { "$date": "2026-10-09T18:30:00Z" }
}
```

4. Embutir ou referenciar?

Decisão 1 — `attendance_records.details` embutido

As informações complementares da ausência são pequenas, pertencem somente àquela ocorrência e normalmente são lidas junto com a frequência. Por isso, `source` e `note` ficam embutidos em `details` . Não há necessidade de consultar outra collection para mostrar a observação da falta.

Essa decisão atende melhor à operação frequente **“registrar uma nova ausência”** , pois o lançamento pode ser gravado em um único documento.

Decisão 2 — aluno, matrícula, responsável e ocorrência referenciados

Os dados principais são referenciados por `ObjectId` porque mudam de forma independente e são reutilizados em muitas consultas. Um aluno possui várias frequências, uma turma possui vários alunos e um responsável pode estar ligado a mais de um aluno. Embutir tudo em um aluno faria os documentos crescerem e dificultaria o histórico de mudança de turma.

Essa decisão atende às operações “**localizar aluno com faltas e responsáveis**” e “**gerar dashboard**”, usando `$lookup` ou agregações controladas.

Decisão 3 — eventos da notificação embutidos em `email_notifications.events`

Os eventos de uma notificação normalmente são poucos, pertencem exclusivamente àquela notificação e são exibidos junto com seu histórico. Por isso, `SENT`, `DELIVERED`, `OPENED` e `FAILED` podem ser embutidos como subdocumentos.

O array não deve crescer sem limite: deve haver política de retenção, limite de reenvios e, se a auditoria exigir muitos eventos, a estrutura deve migrar para `audit_events`, uma collection separada. O conteúdo do e-mail e o endereço usado permanecem no documento principal como snapshot de auditoria.

5. Teste das três consultas mais frequentes

Consulta frequente 1 — localizar aluno, faltas e responsáveis

Pergunta: ao selecionar um aluno na tela de disparo, quais ausências e responsáveis com e-mail válido devem aparecer?

Collections consultadas: `students`, `enrollments`, `attendance_records`, `student_guardians` e `guardians`.

Quantidade de buscas: uma chamada à API com uma aggregation pipeline contendo `$lookup` ou, alternativamente, várias buscas coordenadas no backend.

JavaScript

```
db.students.aggregate([
  { "$match": { "_id": { "$oid": "507f1f77bcf86cd799439011" }, "active": true } },
  { "$lookup": {
    "from": "enrollments",
    "localField": "_id",
    "foreignField": "student_id",
    "as": "enrollments"
  } },
])
```

```

    { "$lookup": {
      "from": "student_guardians",
      "localField": "_id",
      "foreignField": "student_id",
      "as": "links"
    } }
  ] )

```

Depois dos vínculos, a aplicação busca `attendance_records` pelos IDs das matrículas e `guardians` pelos IDs dos vínculos. No código Flask atual, isso é encapsulado pela rota:

Plain Text

```
GET /api/notification-context?student_id=507f1f77bcf86cd799439011
```

A resposta traz `student`, `absences` e `guardians` para a tela preencher os campos automaticamente.

Consulta frequente 2 — registrar ausência

Pergunta: como registrar uma falta para uma matrícula em uma data?

Collection gravada: `attendance_records` .

Quantidade de buscas/gravações: uma validação da matrícula e uma gravação, com índice único para impedir repetição.

JavaScript

```

db.attendance_records.insertOne({
  "enrollment_id": { "$oid": "507f1f77bcf86cd799439021" },
  "date": { "$date": "2026-10-09T00:00:00Z" },
  "status": "ABSENT",
  "registered_by": { "$oid": "507f1f77bcf86cd799439091" },
  "details": {
    "source": "INTERNAL_MESSAGE",
    "note": "Ausência informada pela professora."
  }
} )

```

Índice necessário:

JavaScript

```

db.attendance_records.createIndex(
  { "enrollment_id": 1, "date": 1 },

```

```
{ "unique": true }  
)
```

Consulta frequente 3 — gerar indicadores do dashboard

Pergunta: quantos alunos faltaram por turma e qual é a média de ausências no período?

Collections consultadas: attendance_records , enrollments e classes .

Quantidade de buscas: uma aggregation pipeline com \$lookup , \$match , \$group e \$addToSet .

JavaScript

```
db.attendance_records.aggregate([  
  { "$match": {  
    "status": { "$in": ["PRESENT", "ABSENT"] },  
    "date": {  
      "$gte": { "$date": "2026-10-01T00:00:00Z" },  
      "$lte": { "$date": "2026-10-31T23:59:59Z" }  
    }  
  } },  
  { "$lookup": {  
    "from": "enrollments",  
    "localField": "enrollment_id",  
    "foreignField": "_id",  
    "as": "enrollment"  
  } },  
  { "$unwind": "$enrollment" },  
  { "$lookup": {  
    "from": "classes",  
    "localField": "enrollment.class_id",  
    "foreignField": "_id",  
    "as": "class"  
  } },  
  { "$unwind": "$class" },  
  { "$group": {  
    "_id": "$class._id",  
    "class_name": { "$first": "$class.name" },  
    "absent_students": {  
      "$addToSet": {  
        "$cond": [  
          { "$eq": ["$status", "ABSENT"] },  
          "$enrollment.student_id",  
          null  
        ]  
      }  
    }  
  } },  
  { "$sort": { "absent_students": -1 } }  
])
```

```

    "absence_occurrences": {
      "$sum": { "$cond": [{ "$eq": ["$status", "ABSENT"] }, 1, 0] }
    },
    "present_count": {
      "$sum": { "$cond": [{ "$eq": ["$status", "PRESENT"] }, 1, 0] }
    },
    "valid_attendance_records": { "$sum": 1 }
  } },
  { "$project": {
    "class_name": 1,
    "distinct_absent_students": {
      "$size": {
        "$filter": {
          "input": "$absent_students",
          "as": "student",
          "cond": { "$ne": ["$$student", null] }
        }
      }
    }
  },
  "absence_occurrences": 1,
  "presence_percent": {
    "$multiply": [
      { "$divide": ["$present_count", "$valid_attendance_records"] },
      100
    ]
  }
} }
])

```

A mesma pipeline pode agrupar por `%Y-%m` ou `%Y` para os gráficos de média mensal e anual. O uso de `$addToSet` evita que o mesmo aluno seja contado duas vezes como faltante distinto.

6. Conferência final do modelo

O JSON é válido?

Sim. Os exemplos usam Extended JSON, em que `ObjectId` aparece como `{"_id": "..."} e datas aparecem como {"date": "..."} . As chaves e os valores estão entre aspas quando o formato exige JSON.`

As consultas frequentes usam poucas buscas?

Sim. A tela de notificação usa uma chamada de contexto e uma agregação com referências. O registro de ausência é uma gravação na collection correta, protegida por índice único. O

dashboard usa uma pipeline agregada, em vez de várias consultas por aluno.

Existe array sem limite?

O único array de subdocumentos deste modelo é `email_notifications.events`. Ele deve conter poucos eventos por notificação. Se crescer por reenvios ou auditoria detalhada, deve ser separado em `audit_events`.

Há dados repetidos?

O nome do aluno e o e-mail cadastral não são repetidos como fonte principal. A notificação guarda `recipient_email` e `body` como snapshot do que foi efetivamente enviado. Isso é repetição intencional para auditoria: se o responsável mudar de e-mail depois, o histórico não muda.

É apenas uma cópia do modelo relacional?

Não. O modelo foi organizado pelas operações mais frequentes:

- `details` fica embutido na frequência porque é pequeno e lido junto;
- `events` fica embutido na notificação porque pertence ao histórico daquela mensagem;
- aluno, matrícula, turma e responsável ficam referenciados porque são reutilizados, mudam separadamente e precisam preservar histórico;
- o dashboard usa agregações para responder às consultas analíticas sem duplicar alunos.

7. Conclusão

O modelo combina documentos embutidos para dados pequenos e pertencentes a uma única operação com referências para entidades reutilizadas e históricas. Dessa forma, a aplicação consegue localizar um aluno e seus dados de comunicação, registrar ausências sem duplicidade, receber justificativas por link e gerar indicadores confiáveis no dashboard.